

Design a Shopping Cart

Amazon • Flipkart • Myntra • ప్రతి e-commerce site – "ఒక Map, sku నుంచి qty కి" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది login అప్పుడు – తేసేసిన వస్తువు తిరిగి వస్తుంది, మరియు ఒక retry **cart** ని రెట్టింపు చేస్తుంది. రెండోది ₹850 ని ₹920 చేసి చెప్పదు. మూడోది customer కి 37% సార్లు తక్కువ తగ్గింపు ఇస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

DEEP DIVE 11 (FOOD DELIVERY) చదివారా? – ఇది ఆ బిల్లు కథ కాదు

Food Delivery lo **charge నియమాల జాబితా** చూశాం, మరియు వాటి క్రమం ఒక business నిర్ణయం అని చెప్పాం. అది ఇక్కడ మళ్ళీ చెప్పను.

Shopping cart యొక్క నిజమైన problem పూర్తిగా వేరేది, మరియు అది ఒక్క వాక్యంలో ఉంది:

ఒక order ఒక గత సంఘటన. ఒక cart ఒక సజీవ, పంచుకున్న స్థితి.

Order ఒకసారి ఇచ్చాక మారదు (DD 11 §5). కానీ ఒక cart రెండు ఫోన్లలో, ఒక laptop lo, ఒక guest session lo ఒకేసారి ఉంటుంది – మరియు అవన్నీ కలవాలి. అదే §4.

మరియు §11 lo promotions – DD 11 §12 lo క్రమాన్ని మనం ఎంచుకున్నాం; ఇక్కడ ఉత్తమ క్రమాన్ని వెతకాలి.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు cart ఎక్కడ ఉంటుంది?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: రెండు carts ఒకటవ్వడం

3. Step – `Map<sku, qty>` మరియు ఒక `merge`
4. మొదటి విరుపు – మూడు నియమాలు, మూడూ తప్పు
5. Step – ఎప్పుడు మారిందో గుర్తుపెట్టడం · మరియు ఒక పూర్తి క్రమం

Part 3 – రెండో విరుపు: ధర మారింది, ఎవరూ చెప్పలేదు

6. Step – మొత్తాన్ని ఇప్పటి ధరలతో లెక్కించడం
7. రెండో విరుపు – ₹850 ఎప్పుడో ₹920 అయింది
8. Step – "ఏమి మారింది" చెప్పడం

Part 4 – మూడో విరుపు: తగ్గింపులు

9. Step – అత్యధిక తగ్గింపు ఇచ్చేది ముందు
10. మూడో విరుపు – 37% సార్లు customer నష్టపోతాడు
11. Step – ఉపసమితి మరియు క్రమం వెతకడం

Part 5 – పూర్తి system

12. Step – Checkout ఒక ఒప్పందం

13. మొత్తం code · నడిపి చూద్దాం · మూడు బీజగణిత ధర్మాలు

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటితో చెప్పాల్సిన English script

16. Follow-ups – నిల్వ, save for later, abandoned carts

17. ఏమి నేర్చుకున్నాం

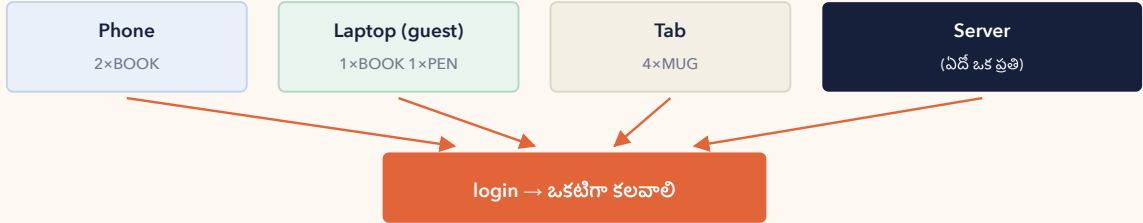
Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు cart ఎక్కడ ఉంటుంది?

"Design a shopping cart. Users add and remove items, see a total, and check out."

ఒక CART · ఎన్ని ప్రతులు?

"User యొక్క cart" – అది ఒకే ఒక వస్తువా?



ఇక్కడ ఒక ప్రశ్న దాగి ఉంది, మరియు అది ఈ problem ని నిర్వచిస్తుంది

ఈ నాలుగు ఒకే cart యొక్క ప్రతులు – ఒక్కొక్కటి వేరే సమయంలో మారినవి.

వాటిని కలపడం ఒక UI పని కాదు – అది ఒక distributed state సమస్య.

మరియు సహజమైన ప్రతి పరిష్కారమూ దాన్ని తప్పుగా చేస్తుంది (§4).

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Guest cart ఉందా? Login అప్పుడు ఏమవుతుంది?	ఇదే మొత్తం problem – §4

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఒకే user, పలు పరికరాలు?	అవును అంటే merge కేవలం login అప్పుడే కాదు – ఎప్పుడైనా
Cart lo ధర ఎప్పటిది – చేర్చినప్పటిదా, ఇప్పటిదా?	§7 – మరియు జవాబు "ఇప్పటిది", కానీ
Cart lo వస్తువు reserve అవుతుందా?	సాధారణంగా కాదు – §16
ఎన్ని promotions కలపొచ్చు?	§11 – ఇది ఒక వెతుకులాట problem
Cart ఎంతకాలం ఉంటుంది?	30-90 రోజులు సాధారణం → ధర మారడం ఖాయం (§7)
వస్తువు అమ్మకం ఆపేస్తే?	§12 – checkout lo చెప్పాలి

మొదటి ప్రశ్న – మరియు దాని వెనక ఉన్న అసలు ప్రశ్న

"User login చేయకుండా cart కట్టాడు. ఇప్పుడు login అయ్యాడు, మరియు అతని ఖాతాలో ఇప్పటికే ఒక cart ఉంది. ఏమి జరగాలి?"

ఇది ఒక UI ప్రశ్నలా అనిపిస్తుంది. కానీ దీని కింద ఒక కఠినమైన ప్రశ్న ఉంది:

"ఈ వస్తువు cart lo లేదు" అంటే ఏమిటి? – ఎప్పుడూ చేర్చలేదా, లేక చేర్చి, ఆపై తీసేశాడా?

Map<sku, qty> lo ఆ రెండూ ఒకేలా కనిపిస్తాయి – key లేదు, అంతే. మరియు ఆ ఒక్క అస్పష్టత §4 lo మూడు వేర్వేరు విధాలుగా విరిగిపోతుంది.

Part 2 – మొదటి విరుపు: రెండు carts ఒకటవ్వడం

3. Step – Map<sku, qty> మరియు ఒక merge

```
class Cart {  
  constructor(owner) { this.owner = owner; this.items = new Map(); } // sku → qty  
  add(sku, qty = 1) { this.items.set(sku, (this.items.get(sku) ?? 0) + qty); }  
  remove(sku) { this.items.delete(sku); }  
  merge(other) { // login సమయంలో  
    for (const [sku, qty] of other.items) this.items.set(sku, qty);  
  }  
}
```

```
guest cart : 2×B00K-1 1×PEN-9  
saved cart : 1×B00K-1 4×MUG-3  
merge తర్వాత: 2×B00K-1 4×MUG-3 1×PEN-9
```

మూడు వస్తువులూ ఉన్నాయి. **పనిచేసినట్టే కనిపిస్తోంది** – కానీ B00K-1 కి saved lo ఉన్న **1 నిశ్శబ్దంగా మాయమైంది**, guest lo ఉన్న 2 గెలిచింది. అది సరైనదేనా?

4. మొదటి విరుపు – మూడు నియమాలు, మూడూ తప్పు

సహజమైన మూడు నియమాలు ఉన్నాయి: **REPLACE** (కొత్తది గెలుస్తుంది), **SUM** (కలపడం), **MAX** (ఎక్కువది). మూడింటినీ మూడు పరిస్థితుల్లో పరీక్షిద్దాం.

సందర్భం 1 – రెండు carts lo ఒకే వస్తువు

```
saved : 1×B00K-1 4×MUG-3  
guest : 2×B00K-1 1×PEN-9  
REPLACE → 2×B00K-1 4×MUG-3 1×PEN-9  
SUM      → 3×B00K-1 4×MUG-3 1×PEN-9  
MAX      → 2×B00K-1 4×MUG-3 1×PEN-9
```

మూడూ సమర్థనీయమే. కానీ:

సందర్భం 2 – login request మళ్ళీ ప్రయత్నించబడింది (retry)
అదే merge రెండేసారి జరిగితే:

REPLACE	ఒకసారి : 2×BOOK-1 4×MUG-3 1×PEN-9	
	రెండుసార్లు: 2×BOOK-1 4×MUG-3 1×PEN-9	✓ ఒకటే
SUM	ఒకసారి : 3×BOOK-1 4×MUG-3 1×PEN-9	
	రెండుసార్లు: 5×BOOK-1 4×MUG-3 2×PEN-9	✗ మారింది
MAX	ఒకసారి : 2×BOOK-1 4×MUG-3 1×PEN-9	
	రెండుసార్లు: 2×BOOK-1 4×MUG-3 1×PEN-9	✓ ఒకటే

సందర్భం 3 – user తన phone lo MUG-3 ని *తీసేశాడు*
కానీ laptop guest session lo అది ఇంకా ఉంది

REPLACE	→ 1×BOOK-1 4×MUG-3	✗ తిరిగి వచ్చింది
SUM	→ 1×BOOK-1 4×MUG-3	✗ తిరిగి వచ్చింది
MAX	→ 1×BOOK-1 4×MUG-3	✗ తిరిగి వచ్చింది

మొదటి విరుపు – మూడు నియమాల్ని విఫలమయ్యాయి, రెండు వేర్వేరు కారణాలతో

SUM idempotent కాదు. Login request ఒకసారి timeout అయి మళ్ళీ ప్రయత్నిస్తే – cart **రెట్టింపు** అవుతుంది.
(Deep Dive 21 §7 lo ATM ని ఇదే సమస్య ₹9,000 debit చేసింది.)

మూడూ తీసేసిన వస్తువుని తిరిగి తెస్తాయి. మరియు ఇది ఏ నియమంతోనూ సరిచేయలేనిది – ఎందుకంటే తప్పు నియమంలో లేదు, **డేటా ఆకారంలో ఉంది.**

మౌలిక తప్పు: Map<sku, qty> lo "MUG-3 లేదు" అనేది **రెండు వేర్వేరు విషయాలని** సూచిస్తుంది – "ఎప్పుడూ చేర్చలేదు" మరియు "తీసేశాను". Merge చేసేటప్పుడు ఆ రెండింటినీ **వేరు చేయడం అసాధ్యం.**

మరియు ఒక లోతైన విషయం: merge చేసేవాడికి **ఏది ఇటీవలిదో కూడా తెలియదు.** Guest cart నిన్నటిదా, ఐదు నిమిషాల క్రితందా?

5. Step – ఎప్పుడు మారిందో గుర్తుపెట్టడం • మరియు ఒక పూర్తి క్రమం

రెండూ కావాలి: **ప్రతి పంక్తికీ ఒక సమయముద్ర**, మరియు **తీసేయడాన్ని ఒక విలువగా** (qty 0) నిల్వ చేయడం – పంక్తిని తొలగించకుండా.

```
// qty 0 = తీసేసినది (tombstone). "లేదు" మరియు "తీసేశాను" వేరు.
const line = (qty, at, by, priceSeen) => ({ qty, at, by, priceSeen });

function merge(a, b) { // తర్వాత మారినది గెలుస్తుంది
  const out = new Map(a);
  for (const [sku, lb] of b) {
    const la = out.get(sku);
    if (!la || lb.at > la.at) out.set(sku, lb);
  }
  return out;
}
```

మూడు సందర్భాలూ సరైనవి:

```
merge : 2×BOOK-1 4×MUG-3 1×PEN-9 ← BOOK-1 కి తాజా విలువ 2
రెండేసారి : 2×BOOK-1 4×MUG-3 1×PEN-9 ✓ ఒకటే
MUG-3 తీసేశాక : 1×BOOK-1 ✓ తీసేసినదే
మళ్ళీ చేర్చితే : 1×BOOK-1 2×MUG-3 ← తాజాది గెలిచింది
```

కానీ – ఇది నిజంగా సరైనదేనా?

ఒక merge function కి **మూడు ధర్మాలు** ఉండాలి:

- **Idempotent** – `merge(merge(a,b), b) = merge(a,b)` . Retry హాని చేయకూడదు.
- **Commutative** – `merge(a,b) = merge(b,a)` . ఏ పరికరం ముందు sync అయిందో పట్టించుకోకుండా ఉండకూడదు.
- **Associative** – `merge(merge(a,b),c) = merge(a,merge(b,c))` . మూడు పరికరాలు ఏ జతలుగా కలిసినా ఒకటే.

20,000 యాదృచ్ఛిక cart త్రయాల మీద పరీక్షిస్తే:

```
20,000 యాదృచ్ఛిక cart త్రయాలు
idempotent · commutative · associative · తాజాది గెలుస్తుంది: x 23
```

Commutative విఫలమైంది – 20,000 లో 23 సార్లు.

కారణం ఒక సమానత్వం

`if (lb.at > la.at)` – అది **కచ్చితంగా ఎక్కువ**. రెండు వంక్తుల సమయముద్రలు **సమానమైతే**, ఈ code మొదటిదాన్ని ఉంచుతుంది.

అంటే `merge(a,b)` లో `a` గెలుస్తుంది, `merge(b,a)` లో `b` గెలుస్తుంది. **క్రమం ఫలితాన్ని మారుస్తుంది** – మరియు అదే commutativity యొక్క ఉల్లంఘన.

Deep Dive 13 §5 లో సమాన scores కి ఇదే సమస్య చూశాం. అక్కడ అది **ప్రదర్శన క్రమాన్ని** అస్థిరం చేసింది; ఇక్కడ అది **cart విషయాన్నే** మారుస్తుంది.

తెగతెంపు ఒక పూర్తి క్రమం కావాలి

```
// పూర్తి క్రమం: సమయం → పరికరం → పరిమాణం. ఏ రెండు వంక్తులకైనా ఒకే జవాబు.
const newer = (x, y) =>
  x.at !== y.at ? (x.at > y.at ? x : y) :
  x.by !== y.by ? (x.by > y.by ? x : y) :
  (x.qty >= y.qty ? x : y);
```

మొదట నేను **పరికరం id** మాత్రమే తెగతెంపుగా పెట్టాను – మరియు అది **216 సార్లు** విఫలమైంది. కారణం: ఒకే పరికరం, ఒకే క్షణం, రెండు వేర్వేరు పరిమాణాలు ఉండొచ్చు. తెగతెంపు **పూర్తిగా** ఉండాలి – ప్రతి జతకీ ఒక నిర్ణయాత్మక జవాబు, అవసరమైతే `qty` వరకు వెళ్ళి.

```
20,000 యాదృచ్ఛిక cart త్రయాలు · 691 సమయ ties
idempotent : ✓
```

commutative : ✓
associative : ✓

ఇది ఒక CRDT - మరియు ఆ పేరు చెప్పడం విలువైనది

ఈ మూడు ధర్మాల్నా ఉన్న ఒక నిర్మాణానికి ఒక పేరు ఉంది: **CRDT** (Conflict-free Replicated Data Type). ఇక్కడ మనం కట్టినది ఒక **LWW-Element-Map** – last-write-wins, tombstones తో.

దీని విలువ: **ఎన్ని ప్రతులైనా, ఏ క్రమంలోనైనా, ఎన్నిసార్లైనా కలపొచ్చు** – ఫలితం ఒకటే. అంటే sync logic lo "ఎవరు ముందు?" అనే ప్రశ్నే అవసరం లేదు.

దీని ధర కూడా ఉంది: LWW అంటే ఒక మార్పు పోతుంది. Phone lo 3, laptop lo 5 అని ఒకే క్షణంలో పెడితే – ఒకటే మిగులుతుంది, రెండూ కాదు. Cart కి అది సరిపోతుంది; ఒక సహకార వత్తానికి సరిపోదు (అక్కడ operation-based CRDT కావాలి).

Part 3 – రెండో విరుపు: ధర మారింది, ఎవరూ చెప్పలేదు

6. Step – మొత్తాన్ని ఇప్పటి ధరలతో లెక్కించడం

Cart lo ధరలు నిల్వ చేయకూడదు – అది Deep Dive 11 §4 యొక్క తప్పు తలకిందులు. Cart ఒక గత సంఘటన కాదు; అది ఒక ఉద్దేశం. కాబట్టి ధర ఎప్పటికప్పుడు చూడాలి:

```
get total() {  
  let t = 0;  
  for (const [sku, q] of this.items) t += this.book.price(sku) * q;  
  return t;  
}
```

ఇది సరైనదే. Flipkart lo cart తెరిస్తే మీకు ఇప్పటి ధరలు కనిపిస్తాయి, మీరు చేర్చినప్పటివి కావు.

7. రెండో విరుపు – ₹850 ఎప్పుడో ₹920 అయింది

సోమవారం – cart lo చేర్చాడు
1xB00K-1 @₹450 • 2xMUG-3 @₹200 → ₹850

మంగళవారం – అమ్మకందారు ధర పెంచాడు (450 → 520)
User cart తెరిస్తే:
1xB00K-1 @₹520 • 2xMUG-3 @₹200 → ₹920

→ ₹850 అనుకున్నది ₹920 అయింది. ఏ సందేశమూ లేదు.
User "checkout" నొక్కితే ₹920 వసూలు అవుతుంది – అతను గమనించకపోవచ్చు.

రెండో విరుపు – ధర సరైనది, ప్రవర్తన కాదు

₹920 అనేది సరైన సంఖ్య. ఇది ఒక లెక్క తప్పు కాదు.

తప్పు ఏమిటంటే – **system కి ఈ మార్పు గురించి తెలుసు, మరియు అది చెప్పలేదు.** User ₹850 అనుకుని cart పెట్టాడు; ₹920 చెల్లిస్తున్నాడు; మరియు ఆ ₹70 ఎక్కడి నుంచి వచ్చిందో అతనికి కనిపించదు.

Carts వారాలు, నెలలు ఉంటాయి. ధర మారడం అరుదైన సంఘటన కాదు – అది **ఖాయం**.

మౌలిక తప్పు: "ఇప్పటి ధర" అనేది ఒక్క సంఖ్య కాదు – అది ఒక **జత**: ఇప్పటి ధర మరియు user చివరిసారి చూసిన ధర. రెండోదాన్ని నిల్వ చేయకపోతే, **మార్పు అనేదే కనిపించదు.**

8. Step – "ఏమి మారింది" చెప్పడం

పరిష్కారం: ప్రతి పంక్తిలో **user చివరిసారి చూసిన ధర** కూడా ఉంచడం – `priceSeen`. ఆపై cart చూపించేటప్పుడు **తేడాలని లెక్కించడం**:

```
// cart ని ఇప్పటి ధరలతో చూపించడం – మరియు *ఏమి మారిందో* చెప్పడం
view(cart) {
  const lines = [], changes = [], unavailable = [];
  let subtotal = 0;
  for (const [sku, l] of cart.entries) {
    if (!this.book.has(sku)) { unavailable.push(sku); continue; }
    const now = this.book.price(sku);
    if (l.priceSeen !== null && l.priceSeen !== now)
      changes.push({ sku, was: l.priceSeen, now,
                    delta: (now - l.priceSeen) * l.qty });
    lines.push({ sku, qty: l.qty, price: now, amount: now * l.qty });
    subtotal += now * l.qty;
  }
  ...
  return { lines, subtotal, unavailable, changes, ...,
          needsConfirm: changes.length > 0 || unavailable.length > 0 };
}
```

```
సోమవారం total      : ₹850
మంగళవారం           : ₹920 • మార్పులు: [ { sku: 'BOOK-1', was: 450, now: 520, delta: 70 }
]
checkout            : { ok: false, reason: 'PRICE_CHANGED',
                       changes: [ { sku: 'BOOK-1', was: 450, now: 520, delta: 70 } ],
                       unavailable: [], newTotal: 920 }
```

Checkout తిరస్కరించింది – మరియు అది **ఎందుకో, ఎంతో** చెప్పింది. User ఒప్పుకుంటే:

```
// ధర మారినట్టు user ఒప్పుకున్నాక – కొత్త ధరలని "చూసినవి" గా నమోదు
acknowledge(cart, at) {
  for (const [sku] of cart.entries)
    if (this.book.has(sku)) cart.set(sku, cart.qty(sku), at, this.book.price(sku));
  return this.view(cart);
}
```

`priceSeen` ఒక ధర నిల్వకాదు – అది ఒక **రసీదు**: "user ఈ ధర చూశాడు". మొత్తం ఎప్పుడూ **ఇప్పటి ధరలతోనే** లెక్కించబడుతుంది. Deep Dive 11 §5 lo order ధరని గడ్డకట్టింది; ఇక్కడ cart అంగీకారాన్ని గుర్తుపెడుతుంది. **వేర్వేరు విషయాలు, వేర్వేరు కారణాలతో.**

Part 4 – మూడో విరుపు: తగ్గింపులు

9. Step – అత్యధిక తగ్గింపు ఇచ్చేది ముందు

Cart కి పలు promotions వర్తించొచ్చు, కానీ గరిష్టంగా రెండు కలపొచ్చు (సాధారణ నియమం). ఏ రెండు?

సహజమైన జవాబు – అత్యధిక తగ్గింపు ఇచ్చేదాన్ని ముందు తీసుకోవడం:

```
function greedy(subtotal, maxN) {  
  const used = new Set(); let rem = subtotal, tot = 0;  
  for (let k = 0; k < maxN; k++) {  
    let bi = -1, bd = 0;  
    for (let i = 0; i < PROMOS.length; i++) {  
      if (used.has(i)) continue;  
      const d = PROMOS[i].discountOn(rem, subtotal);  
      if (d !== null && d > bd) { bd = d; bi = i; }  
    }  
    if (bi < 0) break;  
    used.add(bi); rem -= bd; tot += bd;  
  }  
  return tot;  
}
```

ఐదు promotions: SAVE10 (10%), FLAT100 (₹500+), FLAT250 (₹1500+), SAVE20 (₹2000+), FLAT60 (₹300+). 20,000 carts మీద కొలుద్దాం.

10. మూడో విరుపు – 37% సార్లు customer నష్టపోతాడు

20,000 carts · గరిష్టంగా 2 promotions

అత్యంత ఓడిన సందర్భాలు : 7,444 (37.2%)
సగటు నష్టం (ఓడినప్పుడు): ₹15.52
గరిష్ట నష్టం : ₹25.00

ఉదాహరణలు:

₹656: అత్యంత FLAT100+FLAT60 = ₹160.00 · ఉత్తమం SAVE10+FLAT100 = ₹165.60
₹1659: అత్యంత FLAT250+SAVE10 = ₹390.90 · ఉత్తమం SAVE10+FLAT250 = ₹415.90
₹757: అత్యంత FLAT100+SAVE10 = ₹165.70 · ఉత్తమం SAVE10+FLAT100 = ₹175.70

మూడో విరుపు - రెండో ఉదాహరణ చూడండి

FLAT250+SAVE10 = ₹390.90 · SAVE10+FLAT250 = ₹415.90

అవి అదే రెండు promotions. మారినది కేవలం క్రమం, మరియు తేడా ₹25.

కారణం సూటిది: శాతపు తగ్గింపు, మిగిలిన మొత్తం మీద లెక్కించబడుతుంది. ₹250 ముందు తీసేస్తే, 10% అనేది ₹1409 మీద (₹140.90). 10% ముందు తీసుకుంటే అది ₹1659 మీద (₹165.90).

మరియు అత్యుత్తమ ఎప్పుడూ తప్పు దారి ఎంచుకుంటుంది - ఎందుకంటే ₹250 > ₹165.90, కాబట్టి "అత్యధికం ముందు" అనే నియమం flat ని ముందు పెడుతుంది.

మౌలిక తప్పు: ఇక్కడ రెండు ఎంపికలు ఉన్నాయి - ఏవి (ఉపసమితి) మరియు ఏ క్రమంలో. అత్యుత్తమ రెండింటినీ ఒకే అడుగులో, స్థానికంగా నిర్ణయిస్తుంది. రెండూ కలిసి ఫలితాన్ని నిర్ణయిస్తాయి, కాబట్టి స్థానిక నిర్ణయం సరిపోదు.

11. Step - ఉపసమితి మరియు క్రమం వెతకడం

ఎంపికల సంఖ్య చిన్నది (5 promotions, గరిష్టంగా 2 → 20 అమరికలు), కాబట్టి అన్నింటినీ ప్రయత్నించడమే సరైన జవాబు:

```
// ఉత్తమ ఉపసమితి *మరియు* ఉత్తమ క్రమం - రెండూ వెతకాలి
function bestPromos(subtotal, promos, maxCount = 2) {
  let best = { total: 0, codes: [] };
  const used = new Set(), chosen = [];
  const go = (remaining, total) => {
    if (total > best.total) best = { total, codes: chosen.map(p => p.code) };
    if (chosen.length === maxCount) return;
    for (let i = 0; i < promos.length; i++) {
      if (used.has(i)) continue;
      const d = promos[i].discountOn(remaining, subtotal);
      if (d === null) continue;
      used.add(i); chosen.push(promos[i]);
      go(remaining - d, total + d);
      chosen.pop(); used.delete(i);
    }
  };
  go(subtotal, 0);
  return best;
}
```

"అన్నిటినీ ప్రయత్నించడం" ఇక్కడ ఎందుకు సరైన జవాబు

సాధారణంగా brute force ఒక తప్పు సంకేతం. ఇక్కడ కాదు, మరియు ఎందుకే చెప్పడం ముఖ్యం:

P promotions, గరిష్టంగా **k** – అమరికల సంఖ్య $P!/(P-k)!$. $P=5$, $k=2$ అంటే **20**. $P=20$, $k=3$ అయినా **6,840** – ఒక checkout కి అది ఏమీ కాదు.

మరియు `discountOn null` ఇచ్చిన శాఖలు వెంటనే కత్తిరించబడతాయి, కాబట్టి ఆచరణలో ఇంకా తక్కువ.

k పెరిగితే (అపరిమిత stacking) ఇది పేలుతుంది – అప్పుడు DP లేదా heuristic కావాలి. కానీ **నిజమైన promotion నియమాలు దాదాపు ఎప్పుడూ k ని 1-3 కి పరిమితం చేస్తాయి**, సరిగ్గా ఈ కారణంగానే.

మరియు అది **నిజంగా ఉత్తమమేనా** – ఒక స్వతంత్ర brute force తో పోల్చాను:

20,000 carts · bestPromos = brute force? అవును ✓
అత్యంత ఓడినవి : 7,444 (37.2%) · సగటు నష్టం ₹15.52 · గరిష్టం ₹25.00

ఒక నిజాయితీ గమనిక: నేను ఇక్కడ **customer కి గరిష్ట తగ్గింపు** ని optimise చేస్తున్నాను. కొన్ని సంస్థలు వ్యతిరేకంగా – **కనిష్ట తగ్గింపు** – చేస్తాయి, మరియు కొన్ని క్రమాన్ని **విధానంగా స్థిరపరుస్తాయి** ("శాతాలు ముందు, flat తర్వాత").

ఏ దిశలో optimise చేసినా వాదన ఒకటే: ఫలితం ఉపసమితి మరియు క్రమం మీద ఆధారపడుతుంది, కాబట్టి దాన్ని **వెతకాలి** – ఒక అత్యంత నియమంతో ఊహించకూడదు.

Part 5 – పూర్తి system

12. Step – Checkout ఒక ఒప్పందం

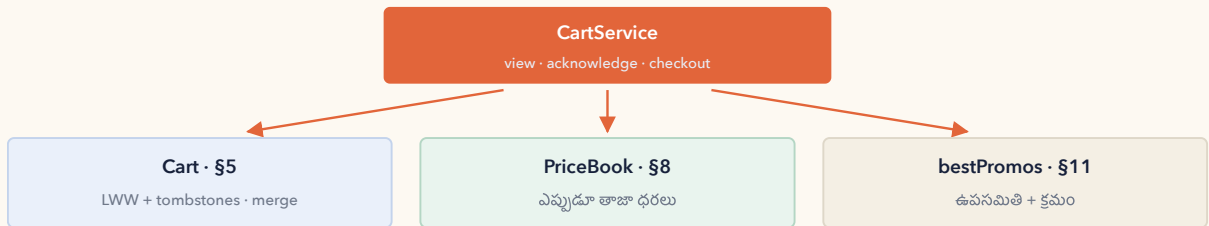
```
checkout(cart, at, { confirmed = false } = {}) {  
  if (cart.isEmpty) return { ok: false, reason: 'EMPTY_CART' };  
  const v = this.view(cart);  
  if (v.needsConfirm && !confirmed)  
    return { ok: false, reason: 'PRICE_CHANGED', changes: v.changes,  
            unavailable: v.unavailable, newTotal: v.total };  
  return { ok: true, ...v, at };  
}
```

needsConfirm రెండు కారణాలని కలుపుతుంది – ధర మారింది, లేదా ఒక వస్తువు అమ్మకంలో లేదు. రెండూ "user చూసినది ఇప్పుడు నిజం కాదు" అని అర్థం, మరియు రెండింటికీ ఒకే ప్రతిస్పందన: **ఆగి, చూపించి, అడగడం**.

13. మొత్తం code · నడిపి చూద్దాం · మూడు బీజగణిత ధర్మాలు

నిర్మాణం · CART ఒక CRDT, మిగతావి దాని చుట్టూ

Cart కి ధర్మాలు తెలియవు · CartService కి merge తెలియదు



Cart ఒక్కటే పంచుకున్న స్థితి – అందుకే దానికే మూడు బీజగణిత ధర్మాలు కావాలి.
ధర్మాలు మరియు promotions ఉత్పన్నమైనవి – అవి ఎప్పుడైనా తిరిగి లెక్కింపబడతాయి.

```
'use strict';  
// qty 0 = తీసేసినది (tombstone). "లేదు" మరియు "తీసేశాను" వేరు.  
const line = (qty, at, by, priceSeen) => ({ qty, at, by, priceSeen });  
  
// పూర్తి క్రమం: సమయం → పరికరం → పరిమాణం. ఏ రెండు పంక్తులకైనా ఒకే జవాబు.  
const newer = (x, y) =>  
  x.at !== y.at ? (x.at > y.at ? x : y) :  
  x.by !== y.by ? (x.by > y.by ? x : y) :  
  (x.qty >= y.qty ? x : y);  
  
class Cart {
```

```

#lines = new Map(); // sku → line
constructor(owner, device = 'web') { Object.assign(this, { owner, device }); }

set(sku, qty, at, priceSeen = null) {
  if (qty < 0) return { ok: false, reason: `BAD_QTY: ${qty}` };
  const cur = this.#lines.get(sku);
  const next = line(qty, at, this.device, priceSeen ?? cur?.priceSeen ?? null);
  this.#lines.set(sku, cur ? newer(cur, next) : next);
  return { ok: true, qty: this.#lines.get(sku).qty };
}

add(sku, qty, at, priceSeen) {
  const cur = this.#lines.get(sku);
  return this.set(sku, (cur?.qty ?? 0) + qty, at, priceSeen);
}

remove(sku, at) { return this.set(sku, 0, at); } // tombstone, తొలగింపు కాదు

get entries() { return [...this.#lines].filter(([, l]) => l.qty > 0); }
qty(sku) { return this.#lines.get(sku)?.qty ?? 0; }
get raw() { return new Map(this.#lines); }
get isEmpty() { return this.entries.length === 0; }

// రెండు carts ని కలపడం – idempotent · commutative · associative
static merge(a, b, owner = a.owner, device = a.device) {
  const out = new Cart(owner, device);
  for (const [sku, l] of a.raw) out.#lines.set(sku, l);
  for (const [sku, lb] of b.raw) {
    const la = out.#lines.get(sku);
    out.#lines.set(sku, la ? newer(la, lb) : lb);
  }
  return out;
}

class PriceBook {
  #p = new Map();
  constructor(init = {}) {
    for (const [k,v] of Object.entries(init)) this.#p.set(k, v);
  }
  price(sku) { return this.#p.get(sku); }
  set(sku, v) { this.#p.set(sku, v); return this; }
  has(sku) { return this.#p.has(sku); }
}

class Promo {
  constructor(code, minSubtotal, fn) {
    Object.assign(this, { code, minSubtotal, fn });
  }
  discountOn(remaining, subtotal) {
    if (subtotal < this.minSubtotal) return null;
    return Math.min(this.fn(remaining, subtotal), remaining);
  }
}

// ఉత్తమ ఉపసమితి *మరియు* ఉత్తమ క్రమం – రెండూ వెతకాలి
function bestPromos(subtotal, promos, maxCount = 2) {
  let best = { total: 0, codes: [] };
  const used = new Set(), chosen = [];
  const go = (remaining, total) => {
    if (total > best.total) best = { total, codes: chosen.map(p => p.code) };
    if (chosen.length === maxCount) return;
  }
}

```

```

    for (let i = 0; i < promos.length; i++) {
      if (used.has(i)) continue;
      const d = promos[i].discountOn(remaining, subtotal);
      if (d === null) continue;
      used.add(i); chosen.push(promos[i]);
      go(remaining - d, total + d);
      chosen.pop(); used.delete(i);
    }
  };
  go(subtotal, 0);
  return best;
}

class CartService {
  constructor(book, promos = [], { maxPromos = 2 } = {}) {
    Object.assign(this, { book, promos, maxPromos });
  }
  // cart ని ఇప్పటి ధరలతో చూపించడం - మరియు *ఏమి మారిందో* చెప్పడం
  view(cart) {
    const lines = [], changes = [], unavailable = [];
    let subtotal = 0;
    for (const [sku, l] of cart.entries) {
      if (!this.book.has(sku)) { unavailable.push(sku); continue; }
      const now = this.book.price(sku);
      if (l.priceSeen !== null && l.priceSeen !== now)
        changes.push({ sku, was: l.priceSeen, now,
                      delta: (now - l.priceSeen) * l.qty });
      lines.push({ sku, qty: l.qty, price: now, amount: now * l.qty });
      subtotal += now * l.qty;
    }
    const promo = bestPromos(subtotal, this.promos, this.maxPromos);
    return { lines, subtotal, unavailable, changes,
            discount: +promo.total.toFixed(2), promos: promo.codes,
            total: +(subtotal - promo.total).toFixed(2),
            needsConfirm: changes.length > 0 || unavailable.length > 0 };
  }
  // ధర మారినట్లు user ఒప్పుకున్నాక - కొత్త ధరలని "చూసినవి" గా నమోదు
  acknowledge(cart, at) {
    for (const [sku] of cart.entries)
      if (this.book.has(sku))
        cart.set(sku, cart.qty(sku), at, this.book.price(sku));
    return this.view(cart);
  }
  checkout(cart, at, { confirmed = false } = {}) {
    if (cart.isEmpty) return { ok: false, reason: 'EMPTY_CART' };
    const v = this.view(cart);
    if (v.needsConfirm && !confirmed)
      return { ok: false, reason: 'PRICE_CHANGED', changes: v.changes,
              unavailable: v.unavailable, newTotal: v.total };
    return { ok: true, ...v, at };
  }
}

```

```

--- $5: merge (LWW + tombstones) ---
saved          : 1×BOOK-1 4×MUG-3
guest          : 2×BOOK-1 1×PEN-9
merge          : 2×BOOK-1 4×MUG-3 1×PEN-9

```

retry తర్వాత : 2×BOOK-1 4×MUG-3 1×PEN-9 ← మారలేదు ✓
MUG-3 తీసేశాక : 2×BOOK-1 1×PEN-9 ← తిరిగి రాలేదు ✓

--- §8: ధర మారితే ---

సోమవారం total : ₹850

మంగళవారం : ₹920 • మార్పులు: [{ sku: 'BOOK-1', was: 450, now: 520, delta: 70 }]

checkout : { ok: false, reason: 'PRICE_CHANGED', changes: [...], newTotal: 920 }

ఒప్పుకున్నాక : { ok: true, subtotal: 920, changes: [], total: 920, ... }

--- §11: promotions - ఉపసమితి *మరియు* క్రమం ---

subtotal ₹1659 → తగ్గింపు ₹415.9 • SAVE10 + FLAT250

చెల్లించేది ₹1243.1

(అత్యాశ FLAT250+SAVE10 ఇచ్చేది ₹390.90 మాత్రమే)

--- తిరస్కరణలు ---

ఖాళీ cart : { ok: false, reason: 'EMPTY_CART' }

ఋణాత్మక qty : { ok: false, reason: 'BAD_QTY: -1' }

అమ్మకంలో లేనిది : { ok: false, reason: 'PRICE_CHANGED', changes: [], unavailable: ['GONE'], newTotal: 100 }

మరియు అసలు రుజువు

20,000 యాదృచ్ఛిక cart త్రయాలు • 726 సమయ ties

idempotent : ✓

commutative : ✓

associative : ✓

20,000 carts • bestPromos = brute force? అవును ✓

అత్యాశ ఓడినవి : 7,444 (37.2%) • సగటు నష్టం ₹15.52 • గరిష్టం ₹25.00

726 సమయ ties – అంటే తెగతెంపు నిజంగా పరీక్షించబడింది, కేవలం code lo ఉంది అని కాదు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	Map<sku, qty>	మౌలిక అస్థిపంజరం
§4 (విరుపు)	at సమయముద్ర, qty: 0 tombstone	SUM రెట్టింపు చేసింది, మూడూ తీసేసినది తెచ్చాయి
§5	by మరియు qty తెగతెంపులు	Commutativity 23 + 216 సార్లు విఫలమైంది
§7 (విరుపు)	priceSeen	₹850 నిశ్శబ్దంగా ₹920 అయింది
§8	changes, acknowledge	మార్పుని చూపించి, అంగీకారం తీసుకోవడం
§10 (విరుపు)	bestPromos	అత్యాశ 37.2% సార్లు తక్కువ ఇచ్చింది
§11	Brute force తో పోలిక	"ఉత్తమం" అనేది నిరూపించాలి

ఎక్కడ నుంచి	ఎమి చేరింది	ఎందుకు
§12	needsConfirm	ధర మారడం మరియు వస్తువు పోవడం – ఒకే ప్రతిస్పందన

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

CRUD ని వేగంగా చాటుండి – merge మరియు promotions మీదే సమయం

6 నిమిషం	Clarify – guest cart? పలు పరికరాలు? ధర ఎప్పటిది?
5 నిమిషం	Cart CRUD - వేగంగా
15 నిమిషం – Merge	tombstones · LWW · మూడు ధర్మాలు
9 నిమిషం – ధరలు	priceSeen · confirm
10 నిమిషం	promotions · నిల్వ · scale

ఏమి తప్పక చెప్పాలి

1. "Guest cart మరియు saved cart కలిసినప్పుడు ఏమవుతుంది?" మీరే అడగండి (§2) – ఇది interview ని తెరుస్తుంది.
2. "లేడు" మరియు "తీసేశాను" వేరు (§4) – tombstone అవసరం. ఇది 20 సెకన్లు మరియు ఇది కీలకం.
3. మూడు ధర్మాలు పేరుపెట్టి చెప్పండి (§5) – idempotent, commutative, associative. మరియు ఎందుకు ప్రతిదీ కావాలో: retry, sync క్రమం, మూడు పరికరాలు.
4. తెగతెంపు పూర్తిగా ఉండాలి (§5) – "timestamps tie, so I'd break by device id and then something deterministic." ఇది చాలా మంది మిస్ చేస్తారు.
5. Promotions lo ఉపసమితి మరియు క్రమం (§10) – "same two promos in a different order differ by ₹25."

ఏమి వదిలేయాలి

- PriceBook రాయెడ్డు – ఒక Map.
- Promotion రకాలు (BOGO, tiered) – ఒక discountOn interface చాలు.
- CRDT సిద్ధాంతం – పేరు చెప్పి, మూడు ధర్మాలు చెప్పి, ముందుకీ. Vector clocks lo దిగొడ్డు (అడిగితే తప్ప).
- Checkout తర్వాత ఏమవుతుంది – అది Deep Dive 11/21.

15. నీటితో చెప్పాల్సిన English script

"Two questions first. Is there a guest cart, and what happens when someone with a guest cart logs in and already has a saved one? And can the same user have a cart open on two devices at once?

I ask because that decides whether this is a CRUD problem or a distributed-state problem, and it's almost always the second. A cart isn't one object – it's several replicas that have to converge.

The obvious model is a map from SKU to quantity, and merging means copying entries across. That breaks in two separate ways. First, if you merge by summing – which is what most sites do – the merge isn't idempotent, so a retried login request doubles the cart. Second, and this one can't be fixed by choosing a different rule: all of replace, sum and max resurrect deleted items. If I removed something on my phone and my laptop's older session still has it, every one of those rules brings it back.

The reason is that in a map from SKU to quantity, 'this SKU isn't here' means two different things – never added, and deliberately removed – and merge can't tell them apart. So each line carries a timestamp and a quantity, where quantity zero is a tombstone rather than a deletion. Merge takes the later write per SKU. Now removal is a fact that can win, and a re-add later can win over that.

I'd then check three properties rather than assume them: idempotent, so retries are safe; commutative, so it doesn't matter which device syncs first; and associative, so three devices converge regardless of how they pair up. I fuzzed twenty thousand random triples and commutativity failed – because when two timestamps are exactly equal, a strictly-greater comparison keeps whichever came first, so merge of a and b differs from b and a. The fix is a total ordering: timestamp, then device id, then something deterministic. I got that wrong once too – device id alone wasn't enough, because the same device can write twice in the same millisecond. What I've described is an LWW-element-map, which is a CRDT, and the trade-off is that concurrent edits lose one side rather than merging – fine for a cart, not fine for a collaborative document.

On pricing: the cart total should use current prices, not prices captured at add-time. A cart lives for weeks, and showing a stale price you won't honour is worse. But then the price changes silently – I had a cart go from eight hundred fifty to nine hundred twenty with no message. So each line also records the price the user last saw, purely as an acknowledgement, and checkout refuses with the list of changes until the user confirms. Notice this is the opposite of an order, which freezes prices, because an order is a past event and a cart is a pending intention.

Last, promotions. If at most two stack, the obvious approach is to apply the biggest discount first. That's wrong thirty-seven percent of the time in my simulation, and the clearest example is two promotions where only the order differs: a flat two-fifty then ten percent gives three ninety, but ten percent then the flat two-fifty gives four fifteen – because the percentage applies to a larger remaining amount. Greedy always picks the flat one first because it's the bigger single

number. Both the subset and the ordering matter, and with a cap of two or three the search space is tiny – twenty arrangements for five promotions – so I'd just search it exhaustively and verify against brute force."

16. Follow-ups – నిల్వ, save for later, abandoned carts

Follow-up	జవాబు	మారే classes
"Save for later"	ఒక <code>status</code> field (<code>ACTIVE</code> / <code>SAVED</code>) – అదే line lo	<code>line</code>
"Cart lo గరిష్ఠంగా 50 వస్తువులు"	<code>set</code> lo ఒక తనిఖీ	1 పంక్తి
"Tombstones పేరుకుపోతాయి"	పాత tombstones ని cart పూర్తిగా sync అయ్యాక తొలగించడం	Storage
"Cart ఖాళీగా 30 రోజులు"	TTL – Deep Dive 02 యొక్క eviction	0 concepts
"వస్తువు నిల్వ అయిపోతే?"	కింద చూడండి	Inventory layer
"Abandoned cart email"	Cart lo ఇప్పటికే <code>at</code> ఉంది → ఒక query	0
"కీల్డ్ carts"	LWW-map చిన్నది; key-value store సరిపోతుంది	Storage

నిల్వ (inventory) – మరియు cart ఎందుకు reserve చేయకూడదు

"Cart lo ఒక వస్తువు పెడితే అది **reserve అవుతుందా?** – దాదాపు అన్ని సైట్లలో **కాదు**, మరియు అది ఒక ఉద్దేశపూర్వక నిర్ణయం.

Carts వారాలు ఉంటాయి. ప్రతి cart వస్తువుని పట్టుకుంటే, **అమ్మకంలో ఉన్న ప్రతిదీ ఎప్పుడూ "అయిపోయినట్టు"** కనిపిస్తుంది – ఎవరూ కొనకుండానే.

కాబట్టి నిల్వ తనిఖీ checkout వద్దే. మరియు అక్కడ Deep Dive 04 §7 యొక్క `atomic claim` కావాలి – ఇద్దరు ఒకేసారి చివరి వస్తువుని కొనకుండా.

నా `needsConfirm` ఇప్పటికే ఈ ఆకారంలో ఉంది – "unavailable" అనేది ధర మార్పులాంటిదే: user చూసినది ఇప్పుడు నిజం కాదు. టికెట్ల వంటి కొరత వస్తువులకి మాత్రం **HELD స్థితి** (DD 04 §9) సరైనది."

17. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
"లేదు" అస్పష్టమైనది	Tombstones (§5)	Sync, soft delete, config defaults
Merge కి బీజగణిత ధర్మాలు	idempotent/commutative/associative (§5)	CRDTs, replication, Deep Dive 21 §8
తెగతెంపు పూర్తిగా ఉండాలి	at → by → qty (§5)	Deep Dive 13 §5 · ఏ sort ఐనా
సజీవ స్థితి vs గత సంఘటన	Cart vs order (§7)	Deep Dive 11 §5, 15 §8
మార్పుని చూపించడం	priceSeen , changes (§8)	Optimistic locking, ETags
స్థానిక ఎంపిక ప్రపంచ ఫలితాన్ని ఇవ్వదు	Promotions (§10)	Deep Dive 20 §4, 05 §9
"ఉత్తమం" అని చెప్తే నిరూపించాలి	brute force పోలిక (§11)	ప్రతి optimisation

ఒక చివరి అలోచన – ఈ DOC LO నేను రెండుసార్లు తప్పు చేశాను

ఒకటి: merge రాసి, "సరైనది" అనుకుని ముందుకి వెళ్ళబోయాను. Fuzz test **23 commutativity** వైఫల్యాలు చూపించింది – ఒక `>` మరియు `>=` మధ్య తేడా.

రెండోది: దాన్ని "సరిచేసి" పరికరం id తెగతెంపు పెట్టాను. మళ్ళీ నడిపితే **216 వైఫల్యాలు** – ఒకే పరికరం, ఒకే క్షణం అనే సందర్భాన్ని నేను అలోచించలేదు.

రెండుసార్లు నా కంటికి code సరైనదిగానే కనిపించింది. రెండుసార్లు దాన్ని పట్టుకున్నది ఒక ధర్మం – ఒక test case కాదు.

పాఠం: "ఇది సరిగ్గా merge చేస్తుంది" అనేది ఒక అభిప్రాయం. "merge(a,b) = merge(b,a) ఎప్పుడూ" అనేది ఒక తనిఖీ చేయగలిగిన వాదన. Merge, sync, లేదా ఏదైనా కలిపే code రాసినప్పుడు – **సందర్భాలు కాదు, ధర్మాలు రాయండి.**

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library · 16 Tic-Tac-Toe · 17 Autocomplete · 18 Snake & Ladder · 19 Hit Counter · 20 Vending Machine · 21 ATM · 22 Logging · 23 Shopping Cart.**

Deep Dive 11 (Food Delivery) దీని కొనసాగింపు – checkout తర్వాత cart ఒక order అవుతుంది, మరియు అక్కడే అది గడ్డకడుతుంది.

Shopping Cart – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి శాతం నిజంగా `node lo run` చేసి తీసినదే ✓